JENKINS INTERVIEW CHEAT SHEET — 3 YOE DevOps

Keywords & technical terms only — quick-scan before the interview

CORE & ARCHITECTURE

Jenkins: open-source automation server, CI/CD orchestrator, plugin-driven
Controller (Master): scheduling, UI, plugin mgmt, `JENKINS_HOME`, queue mgmt, no build code (0 executors best practice)
Agent (Node/Slave): executes builds; SSH / JNLP connect
Executor: slot on node, 1 build/step at a time
Queue: holds pending jobs until executor free

PIPELINE TYPES

Freestyle vs **Pipeline job**
Declarative: `pipeline{}`, structured, linted, restartFromStage
Scripted: `node{}`, pure Groovy, imperative, flexible loops
Jenkinsfile: pipeline-as-code, SCM-versioned

JENKINSFILE BLOCKS

`agent` · `options` · `parameters` · `environment` · `triggers` · `stages` · `post`

AGENT

`agent any` / `agent none` / `label` / `docker{}` / `kubernetes{}`
pipeline-level vs stage-level override

ENVIRONMENT

global vs stage-scoped
`credentials()` auto-mask
built-ins: `BUILD_NUMBER`, `BUILD_URL`, `JOB_NAME`, `WORKSPACE`, `GIT_COMMIT`

PARAMETERS

types: `string`, `choice`, `booleanParam`, `text`, `password`, `file`

OPTIONS

`timeout()`, `retry()`, `timestamps()`, `disableConcurrentBuilds()`, `buildDiscarder(logRotator())`, `skipDefaultCheckout()`, `ansiColor()`

STAGES

components: `steps`, `when`, `post`, `parallel`, `agent` (stage-level)
sequential top-down, fail = halt (unless caught)

CREDENTIALS STORE

types: Username/Password, SSH key, Secret text, Secret file, Certificate
scope: Global / System / Folder
`withCredentials()`, auto-masking in logs

TRIGGERS

`pollSCM()` (inefficient), `cron()`, `upstream()`, `githubPush()` (webhook — preferred)

WEBHOOK

HTTP POST on push/PR/merge → instant trigger
vs polling: no delay, less load
URL: `/github-webhook/`

SCM

`checkout scm`, `git branch:`, credentialsId

MULTIBRANCH PIPELINE

auto job-per-branch, PR discovery, orphan branch cleanup
`when { branch 'main' }`

BUILD STATUS DETERMINATION

exit code 0 = SUCCESS; non-zero = FAILURE
states: SUCCESS, FAILURE, **UNSTABLE** (tests failed, build ok), ABORTED
`currentBuild.result`

POST CONDITIONS

`always`, `success`, `failure`, `unstable`, `aborted`, `changed`, `fixed`, `regression`, `cleanup` (runs last)

COMMON STAGES

Checkout → Build → Test (`junit`) → Archive (`archiveArtifacts`) → SonarQube → Quality Gate → Docker Build → Trivy Scan → Push ECR → Cosign Sign → Update GitOps → Helm Lint/Template → Deploy → Smoke Test → Notify

SONARQUBE

`withSonarQubeEnv()`, `waitForQualityGate(abortPipeline:true)`
async — needs webhook callback + timeout

QUALITY GATE

thresholds: coverage %, bugs, vulnerabilities, code smells, duplication
→ pass/fail gate before deploy

STATIC VS DYNAMIC ANALYSIS

Static: SonarQube, ESLint, Checkstyle, Semgrep — no execution
Dynamic (DAST): OWASP ZAP, Burp — runtime, needs running app

TRIVY

`trivy image --severity HIGH,CRITICAL --exit-code 1`
scans: images, filesystem, IaC, repo; local CVE DB cache

ECR PUSH

`aws ecr get-login-password` → `docker login` → tag → push
auth: IAM role (preferred) vs static creds
tag with SHA/build number, not `latest`; lifecycle policy cleanup

COSIGN

Sigstore — sign/verify image signatures, supply-chain security
`cosign sign --key` / `cosign verify --key`
blocks tampered/unauthorized images at admission control

GITOPS REPO UPDATE

Jenkins updates manifest/values (not direct deploy) → ArgoCD/Flux syncs
`yq -i '.image.tag'`, git commit + push
decouples CI from CD, no cluster creds in Jenkins

HELM VALIDATE

`helm lint` (chart syntax/best practice)
`helm template` (render w/o apply) + `kubeval`/`kubeconform`

SHARED LIBRARIES

reusable Groovy in separate repo, `@Library('name') _`
`vars/` = global step fns; `src/` = classes
versioned: `@Library('lib@v2.1.0')`

PARALLEL STAGE

`parallel { stage(){} stage(){} }` — independent stages, cuts duration

MATRIX BUILD

`matrix { axes { axis{} } }` — combinations of OS × JDK × env, auto-generated

INPUT / APPROVAL GATES

`input message: ok: submitter:`
types: simple, scoped submitter (RBAC), timed (`timeout{}`), parameterized
caution: holds executor unless `agent none`

ERROR HANDLING

`try/catch`, `catchError(buildResult:,stageResult:)`, `retry()`, `timeout()`, `post{failure{}}`
fail fast (critical) vs record & continue (flaky)

STATIC VM VS K8S AGENTS

Static: always-on, manual scale, shared-state risk
K8s (dynamic): pod-per-build, ephemeral, scale-to-zero, autoscaler, pod templates

WORKSPACE

`$WORKSPACE`, per-job dir, `cleanWs()` post-build
concurrency conflicts → `disableConcurrentBuilds()` / `ws()`
significance: disk mgmt, reproducibility, no stale artifacts

NOTIFICATIONS

`slackSend`, `emailext`, Teams, PagerDuty/Opsgenie
failure/unstable = always notify; success = prod only; use `changed`/`fixed` to reduce noise

SECURITY BEST PRACTICES

credential scoping (folder-level), branch protection on Jenkinsfile, mask secrets, restrict fork-PR agent trust, IAM roles over static keys

SCALING JENKINS

K8s dynamic agents, 0 executors on controller, shared libs for standardization, folder RBAC, credential scoping

DEPLOY STRATEGIES

Blue-Green: deploy parallel, switch service/ingress selector
Canary: gradual traffic shift, Istio/Argo Rollouts, health-gated increments